An IEEE Conference Paper Template for Technical and Research Reports with Modular Local Compilation

Nayaka Ghana Subrata – 13523090[1,2]
Department of Informatics Engineering
School of Electrical Engineering and Informatics
Institut Teknologi Bandung, Jl. Ganesha 10, Bandung 40132, Indonesia
[1]13523090@std.stei.itb.ac.id, [2]nayakaghana39@gmail.com

***Abstract*—This repository provides a reusable IEEE-style paper template for technical reports, research drafts, class assignments, and project documentation. The template follows a modular structure in which the main document defines typography, page style, package selection, bibliography handling, and repeated metadata, while individual sections are stored in separate files. The content in this version is intentionally generic and can be replaced with material from any computing, engineering, data science, systems, or applied research topic. The template includes examples of abstract writing, keyword selection, section organization, equations, algorithms, tables, figure usage, citations, repository disclosure, and academic statement formatting. It is designed to compile locally with XeLaTeX and BibTeX using the bundled IEEEtran class file, so the repository can be used without depending on an external IEEE template download. The dummy article spans approximately six IEEE conference pages and demonstrates the layout density expected from a compact technical paper.**

***Index Terms*—IEEE template, LaTeX, technical report, academic writing, local compilation, modular paper**

I. Introduction

Academic technical papers often require a precise balance between conceptual clarity, implementation detail, and reproducible evaluation. A useful paper template should therefore provide more than an empty title page. It should demonstrate how to organize a technical argument, where to place equations and algorithms, how to cite prior work, and how to keep local compilation predictable across machines. This repository is intended to serve that purpose and follows established LaTeX and IEEEtran practices [1], [2].

The template follows a compact IEEE conference layout. The document is written in two columns, uses compact section headings, includes a simple footer, and stores each major section in a separate file. This modular arrangement makes editing safer because a writer can focus on one part of the paper at a time without scrolling through a large monolithic source file. It also makes version control easier, since changes to the introduction, background, evaluation, or statement appear as separate file diffs.

The subject matter in this template is intentionally generic. Instead of claiming a domain-specific contribution, the paper explains how a technical report can be structured. The examples mention broad research and engineering concerns such as system design, experiment pipelines, reproducibility, measurement, and citation practice. A user can replace them with a concrete topic such as machine learning evaluation, distributed systems, network measurement, human-computer interaction, software engineering, embedded systems, cybersecurity, or applied data analysis.

The template has four practical goals. First, it provides a working IEEEtran-based source tree that compiles locally. Second, it demonstrates a paper length near six pages, which is a realistic target for a short technical assignment. Third, it includes common LaTeX constructs used in technical writing: inline math, displayed equations, pseudocode, result tables, and figures. Fourth, it documents the local build commands directly in the repository so the paper can be regenerated without guessing the toolchain.

The remainder of this paper is organized as follows. Section II describes the visual and structural conventions. Section III explains the file layout. Section IV gives writing guidelines for each paper component. Section V provides technical LaTeX examples. Section VI explains local compilation. Section VII describes how to customize the template for a new paper, and Section VIII concludes.

II. Background

A. IEEE Conference Style

IEEE conference papers use a dense two-column format intended for technical communication. Titles are centered, author metadata is placed below the title, abstracts are compact, and body sections are numbered automatically. The class file `IEEEtran.cls` handles most of these conventions. This template keeps the class file inside the `paper/` directory so compilation does not depend on a global LaTeX installation containing the exact same version.

The reference repositories use XeLaTeX because their paper sources select Times New Roman when available and fall back

to TeX Gyre Termes otherwise. That behavior is preserved here through `fontspec`. If a machine has Times New Roman installed, the output will match common IEEE-style typography closely. If it does not, TeX Gyre Termes provides a compatible serif font for local builds.

B. Technical Paper Expectations

A technical report should normally separate assumptions, design, implementation, and evaluation. The background should define the concepts, systems, datasets, methods, or standards used by the work. The design section should state what is being built or analyzed. The implementation section should describe concrete modules, parameters, data formats, and execution flow. The evaluation section should report measured, simulated, or observed results rather than only restating theory. Even when a paper is short, this separation helps readers distinguish established knowledge from the author's contribution.

Technical notation should be explicit. For example, let x denote an input vector, let f denote a model or system under test, and let y denote an observed output. A simple evaluation interface can be summarized as

$$y \leftarrow f(x; \theta), \tag{1}$$

where θ represents configuration, parameters, or learned weights. A measurement can then compare y against an expected output, baseline, or target metric. This level of notation is usually enough for a systems-oriented paper, while a theory-heavy paper may require more formal definitions.

C. Reproducibility

Local reproducibility matters because a paper source is not complete until it can be compiled again. Reproducible research practice emphasizes keeping source, data, scripts, and documentation organized enough that results can be regenerated later [3], [4]. A repository should contain the main `.tex` file, section files, bibliography file, required figures, and any class files that are not guaranteed to exist in the default installation. Build artifacts such as `.aux`, `.log`, `.bbl`, and `.pdf` are intentionally ignored in this template. They can be regenerated from source and should not obscure meaningful changes in version control.

III. Template Structure

A. Repository Layout

The repository is organized around a `paper/` directory. The top-level `README.md` explains how to compile the document, while `paper/main.tex` defines the global document configuration. Section files live under `paper/sections/`. Bibliographic entries are stored in `paper/references.bib`. Optional figures can be placed in `images/` and referenced from the paper using relative paths.

TABLE I
Main files in the template repository

Path	Purpose
`paper/main.tex`	Main IEEE document, packages, title, author, abstract, and section inputs.
`paper/sections/`	One file per major paper section.
`paper/references.bib`	BibTeX entries and IEEE bibliography control entry.
`paper/IEEEtran.cls`	Local IEEEtran class file used by the document.
`images/`	Optional figure and image assets.
`README.md`	Local compile instructions and editing notes.

B. Main Document

The main document imports packages that are commonly useful in technical papers. The `amsmath` packages support mathematical notation, `algorithm` and `algpseudocode` support pseudocode, `booktabs` supports clean tables, and `hyperref` provides link handling. The template also demonstrates how to include repository images using paths relative to `paper/main.tex`.

The body of `main.tex` should stay short. It should contain metadata and the ordered list of `\input` commands. Most prose belongs in section files. This keeps the document easy to scan and reduces merge conflicts if more than one person edits the paper.

C. Section Files

Each section file starts with a normal `\section` command, except for unnumbered administrative sections such as acknowledgments, repository disclosure, and statement. A typical paper can replace the current generic sections with a domain-specific sequence such as introduction, background, threat model, design, implementation, evaluation, discussion, conclusion, and acknowledgment. The exact order is less important than making the reader's path through the argument clear.

IV. Writing Guidelines

A. Abstract and Keywords

The abstract should state the problem, the method, and the result in one compact paragraph. For a class or project paper, it is acceptable to mention that the work is an implementation, evaluation, replication, or analysis rather than a new method. Avoid writing an abstract that only introduces the topic. A reader should understand what was built, what was measured, and what conclusion the paper reaches.

Keywords should be specific enough to help classification. For example, a paper about model evaluation may use terms such as *benchmarking*, *classification*, *latency*, and *reproducibility*. A paper about a software system may use terms such as *architecture*, *data pipeline*, *performance analysis*, and *deployment*.

B. Problem Statement

A good problem statement should identify the gap or failure mode being addressed. Depending on the topic, the issue may be inaccurate predictions, high latency, poor usability, data loss, unreliable synchronization, weak scalability, deployment friction, or inconsistent measurement. The statement should then explain why the chosen method addresses that issue. This prevents the paper from becoming a general survey with no concrete claim.

C. Design and Implementation

Design sections should describe abstractions and goals. Implementation sections should describe concrete choices. For example, a design section may state that a system separates data ingestion from analysis and reporting. The implementation section should then name the modules, fields, command-line options, and data files used to realize that separation.

When presenting code-level work, avoid pasting long source listings. A short algorithm is useful when it clarifies control flow, but the repository should hold the complete implementation. The paper should explain decisions that are not obvious from code alone: why a parameter was selected, why a data format was chosen, what assumptions are made, and which cases are intentionally rejected.

D. Evaluation

Evaluation should connect back to the original claim. If the claim is performance, report runtime, memory use, bandwidth, or storage overhead. If the claim is correctness, report success and failure cases. If the claim is usability, report task completion, error rate, or qualitative feedback. Even a small experiment is stronger when it has explicit scenarios and expected outcomes.

V. Technical Examples

A. Algorithm Block

Algorithm 1 shows a simple structure for documenting an experiment pipeline. The pseudocode is intentionally generic. Replace the setup, execution, and recording steps with the operations used by the real project.

Algorithm 1 Generic Experiment Pipeline

Require: Scenario set S, trial count r, random seed $seed$
Ensure: Result table R

```
Initialize deterministic randomness using seed
R ← ∅
for all s ∈ S do
    for i ← 1 to r do
        Generate input material for scenario s
        Execute the implementation under test
        Record success flag, runtime, and output size
    end for
end for
return aggregate statistics in R
```

B. Figure Example

Figures should be readable in two-column format. Use vector graphics or high-resolution bitmap images when possible. This example uses `reiko.png` from the repository-level `images/` directory.

Fig. 1. Example repository image included from `images/reiko.png`. Replace this asset with a project-specific figure when preparing the final paper.

C. Two-Column Figure

Wide figures can span both columns by using the `figure*` environment. In IEEE-style layouts, wide floats usually move to the top of the next page, so the text should introduce them before the figure appears.

D. Result Table

Tables should use concise labels and avoid excessive grid lines. Table II illustrates a compact evaluation table. The values are dummy numbers and should be replaced by real measurements.

TABLE II
Example result table for a technical experiment

Scenario	Trials	Success	Median ms
Baseline verification	25	25	1.8
Large-message signing	25	25	7.4
Corrupted input reject	25	25	2.1
Insufficient material	25	0	0.9

E. Two-Column Table

Wide tables can span both columns by using the `table*` environment. This is useful when a comparison has many columns or when descriptions would wrap too aggressively in a single-column table.

F. Equations

Equations should be introduced in prose and referenced when they matter. For instance, an average latency metric can be written as

$$\bar{L} = \frac{1}{n} \sum_{i=1}^{n} L_i, \tag{2}$$

where L_i is the latency observed in trial i and n is the number of trials. Equation (2) is useful only if the text explains how measurements are collected, filtered, and interpreted.

Fig. 2. Example two-column figure included from `images/abcd.jpg`. Use this format for diagrams, screenshots, timelines, or result plots that would be too cramped in a single column.

TABLE III
EXAMPLE TWO-COLUMN COMPARISON TABLE

Variant	Input condition	Metric A	Metric B	Interpretation
Baseline	Default configuration and nominal workload	1.00	74.2 ms	Reference point for the remaining variants.
Optimized	Same workload with tuned parameters	0.92	51.8 ms	Lower latency with similar output quality.
Stress case	Burst workload and larger input size	1.37	116.4 ms	Higher cost under heavier demand.
Fallback	Dependency unavailable or degraded	1.11	88.9 ms	Slower but still operational.

VI. Local Compilation

A. Recommended Toolchain

The recommended local build path is XeLaTeX plus BibTeX. On Windows, this can be installed through MiKTeX or TeX Live. The document uses `fontspec`, so `pdflatex` is not the right compiler for this template. Use `xelatex` instead.

From the repository root, compile with the following commands:

```
cd paper
xelatex main.tex
bibtex main
xelatex main.tex
xelatex main.tex
```

The repeated XeLaTeX passes are intentional. The first pass creates auxiliary citation data, BibTeX creates the bibliography, and the final passes resolve references, citations, and labels. The output PDF will be written to `paper/main.pdf`.

B. Latexmk Alternative

If `latexmk` is installed, the build can be shortened to:

```
cd paper
latexmk -xelatex main.tex
```

To clean generated files while keeping the PDF, run:

```
latexmk -c main.tex
```

To remove generated files including the PDF, run:

```
latexmk -C main.tex
```

C. Tectonic Note

Tectonic can also compile many XeLaTeX documents, but BibTeX workflows may differ by installation and bundle state. If Tectonic is used, verify that citations and the bibliography appear correctly in the final PDF. For this template, the most predictable local path remains XeLaTeX followed by BibTeX and two additional XeLaTeX passes.

D. Common Build Problems

If LaTeX reports that `IEEEtran.cls` is missing, confirm that compilation is executed from the `paper/` directory or that the local class file still exists. If the compiler rejects `fontspec`, the command is probably using `pdflatex`; switch to `xelatex`. If citations appear as question marks, run BibTeX and then rerun XeLaTeX twice. If a figure path fails, check whether paths are relative to `paper/main.tex`; figures in the repository-level `images/` directory should be referenced as `../images/name.png`.

VII. Customization

A. Replacing Metadata

Start customization in `paper/main.tex`. Replace the title, author block, affiliation, email addresses, abstract, and keywords. Keep the `\maketitle` sequence and the page-style commands unless the assignment requires a different layout. The current footer follows the style of the reference repositories and can be edited in the `\fancyfoot` command.

B. Changing Sections

The safest way to create a new paper is to rename or replace section files one at a time. After changing the file list, update the corresponding `\input` commands in `main.tex`. Compile after every few changes. This catches missing braces, unresolved citations, and broken paths early.

C. Adding Figures

Place final image files in the repository-level `images/` directory. From a section file, reference them with a relative path such as:

```
\includegraphics{../images/diagram.png}
```

Keep figure captions descriptive. A caption should explain what the reader should notice, not merely repeat the figure label. If a figure is too wide for one column, IEEEtran supports `figure*`, but use it sparingly because wide floats can move far from the text that introduces them.

D. Managing References

Add references to `paper/references.bib` and cite them with `\cite{key}`. Prefer primary sources for algorithms, standards, protocols, datasets, and tools. For example, cite the original paper for a method, the official standard for a technical specification, and the project documentation for a software dependency. This keeps the bibliography defensible and avoids relying on secondary summaries for technical claims.

E. Adapting the Length

The current draft is sized to demonstrate a short six-page conference-style paper. If an assignment has a strict page limit, adjust length by changing the density of examples rather than by changing margins or font size. IEEE formatting should remain stable. Remove optional examples, shorten tables, or move implementation details into the repository README when the paper is too long. Add evaluation discussion, limitations, and threat-model detail when the paper is too short.

VIII. Review Checklist

A. Before Writing

Before replacing this template with a real topic, define the paper's core claim in one sentence. A claim such as "this implementation reduces median request latency under a fixed workload" is easier to evaluate than a broad theme such as "performance matters." The claim determines which background material is necessary, which experiments should be included, and which results are relevant.

The writer should also identify the expected reader. A classmate may need more implementation context, while a reviewer or instructor may expect clearer assumptions and stronger references. The template does not force one level of detail, but it provides enough structure to support either emphasis.

B. Before Submission

Use the following checks before considering the paper complete:

1) The abstract states the problem, method, and result.
2) Every nontrivial technical claim has a citation, derivation, or experiment.
3) All figures and tables are referenced from the text before or near their placement.
4) All citations resolve and the bibliography appears in IEEE style.
5) The repository link points to the final public or private repository expected by the assignment.
6) The statement section reflects the required academic integrity wording.
7) The PDF was regenerated from a clean local compile, not from a stale build artifact.

This checklist is intentionally practical. Formatting errors are usually easy to fix, but missing claims, unclear assumptions, and unsupported results are harder to repair near a deadline. A short review pass focused on those issues improves the paper more than repeated cosmetic editing.

C. Version Control Hygiene

Generated LaTeX files should not be committed unless the assignment, venue, or project policy explicitly asks for the final PDF in the repository. The source of truth is the combination of `main.tex`, section files, references, figures, and README instructions. Keeping generated files out of commits makes it easier to review the actual writing and prevents large binary diffs from hiding meaningful changes.

When working over multiple sessions, compile before stopping and check the git status. A clean build confirms that the next editing session starts from a known-good source tree. If a section is incomplete, leave a clear placeholder paragraph rather than a broken command or missing file input.

IX. Example Paper Skeleton

A. Implementation-Oriented Paper

An implementation-oriented technical paper can use the following narrative arc. The introduction motivates a concrete operational problem, such as reducing latency, improving reliability, automating a workflow, processing a dataset, or evaluating a model. The background then explains only the concepts needed by the implementation. If the paper uses a machine learning model, it should define the task, input representation, baseline, and metrics. If the paper describes a software system, it should define the components, interfaces, workload, and expected behavior.

The design section should describe the system boundary. For example, a data-processing report can distinguish ingestion, cleaning, transformation, storage, and reporting stages. A web-system report can distinguish client, server, database, cache, and external service boundaries. Clear boundaries make evaluation easier because the paper can state which component is measured and which component is assumed.

The implementation section should map the design to code. It should name the programming language, libraries, modules, data structures, and command-line entry points. A concise module table is often enough. The text should then explain details that affect correctness: input format, preprocessing, error handling, parameter validation, persistence, and test scenario generation. These details are important because many project failures come from integration mistakes rather than from the central algorithm alone.

The evaluation section should be organized around scenarios. A scenario describes the input condition, expected behavior, and observed result. For example, a web application may test normal traffic, burst traffic, invalid input, cache misses, and dependency failure. A model-evaluation paper may test clean data, noisy data, class imbalance, out-of-distribution samples, and baseline comparisons. Each scenario should have a reason to exist.

B. Analysis-Oriented Paper

An analysis-oriented paper can use a different structure. After the introduction and background, it may define an evaluation model, compare several candidate designs, and evaluate them against qualitative criteria. This is appropriate when the work studies architecture choices, deployment tradeoffs, workflow policies, model choices, or system constraints. The key requirement is that the comparison criteria are stated before conclusions are drawn. Criteria may include accuracy, latency, complexity, cost, maintainability, compatibility, failure behavior, and operational risk.

For example, a deployment paper might compare a single-server design, a cached design, a queue-based design, and a replicated design. The analysis should explain what each design improves, what each design complicates, and which failure modes remain. Even if the measurements are simple, the paper becomes useful when it connects design behavior to a concrete user or operator goal.

C. Suggested Section Map

Table IV gives a reusable section map for a six-page technical report. The page allocation is approximate and should be adjusted based on the assignment or venue.

TABLE IV
Approximate six-page paper allocation

Part	Pages	Main purpose
Abstract and intro	0.8–1.0	State problem, method, and contribution.
Background	0.8–1.0	Define concepts and assumptions.
Design	1.0	Explain architecture and goals.
Implementation	1.0	Map design to concrete code and parameters.
Evaluation	1.2–1.5	Report scenarios, measurements, and outcomes.
Conclusion and admin	0.5	Summarize findings, repository, and statement.

This allocation avoids two common problems. The first is an oversized background section that leaves little room for the author's actual work. The second is an evaluation section that reports numbers without explaining why they matter. A balanced paper gives enough theory for context, but reserves space for design decisions, implementation constraints, and evidence.

X. Conclusion

This template provides a complete IEEE-style starting point for a technical paper. It includes the document class, package configuration, author and abstract structure, modular section files, bibliography setup, examples of common technical elements, and local compile instructions. The included content is deliberately replaceable, but it is long enough to demonstrate how a six-page paper behaves under the selected formatting.

The main practice encouraged by the template is separation of concerns. Global formatting belongs in `main.tex`, prose belongs in section files, citations belong in `references.bib`, and generated artifacts should stay out of version control. With that structure in place, a writer can focus on the actual contribution while preserving a consistent paper format.

Acknowledgment

The author thanks reviewers, instructors, collaborators, and peers who provide feedback on technical writing, implementation discipline, and reproducible evaluation. This template was prepared as a reusable starting point for future paper drafts.

References

[1] L. Lamport, *LaTeX: A Document Preparation System*, 2nd ed. Addison-Wesley, 1994.

[2] M. Shell, *How to Use the IEEEtran LaTeX Class*, IEEE, 2015. [Online]. Available: https://mirrors.ctan.org/macros/latex/contrib/IEEEtran/IEEEtran_HOWTO.pdf

[3] R. D. Peng, "Reproducible research in computational science," *Science*, vol. 334, no. 6060, pp. 1226–1227, 2011.

[4] G. Wilson, J. Bryan, K. Cranston, J. Kitzes, L. Nederbragt, and T. K. Teal, "Good enough practices in scientific computing," *PLOS Computational Biology*, vol. 13, no. 6, pp. 1–20, 2017.

Repository

The source files for this IEEE paper template are available in the following repository:

https://github.com/Nayekah/IEEE-Format

Statement

I hereby declare that this paper template and its example text are prepared as a formatting scaffold. Any future paper derived from this template must replace the placeholder content with the author's own work and must cite all external sources used.

Bandung, May 19th 2026

Nayaka Ghana Subrata

NIM 13523090